

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Kraman Zhaniya
Student Identification Number	230103077
Date & Laboratory Session	17.09.2026 02-N
Workstation / Laptop Make & Model	Macbook M3
Processor SKU / SoC Model	e.g., AMD Ryzen 7 5800H / Apple M2 Pro / Intel Core i7-12700H
Core Topology (Physical P-cores / E-cores)	Physical Cores: 8 Cores Logical SMT Threads: 8 Threads
Cache Hierarchy Capacity	L1: 64 KB L2: 20 MB L3: 8MB
Host OS & Python Runtime	e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8

■ **MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE**

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ **EXECUTION PROTOCOL**

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

```
jooke@MacBook-Air--jooke CSS 314 % python3 amdahl_test.py
```

```
Cores | Time(s) | Observed Speedup | Theoretical Linear | Efficiency
```

```

-----
 1 | 2.1561 | 1.00x | 1.00x | 100.00%
 2 | 1.0588 | 2.04x | 2.00x | 101.82%
 4 | 0.5760 | 3.74x | 4.00x | 93.58%
 8 | 0.4453 | 4.84x | 8.00x | 60.52%
12 | 0.4696 | 4.59x | 12.00x | 38.26%
16 | 0.5013 | 4.30x | 16.00x | 26.88%

```

```
jooke@MacBook-Air--jooke CSS 314 %
```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	2.1561	1.00x	1.00x	100.0%
p = 2	1.0588	2.04x	2.00x	101.82%

p = 4	0.5760	3.74x	4.00x	93.58%
p = 8	0.5760	4.84x	8.00x	60.52%
p = 12	0.4696	4.59x	12.00x	38.26%
p = 16	0.5013	4.30x	16.00x	26.88%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

The observed speedup increases up to **8 cores**, where it reaches **4.84x**. After that, the speedup decreases to **4.59x at 12 cores** and **4.30x at 16 cores**.

Therefore, the inflection point is:

$p^* = 8$ cores

My machine has an **Apple M3 CPU with 8 physical cores: 4 Performance (P) cores + 4 Efficiency (E) cores**. It does not use SMT/hyperthreading, so there are **8 logical CPU threads** as well.

The results match this topology: performance improves until all **8 physical cores (4P + 4E)** are used. Increasing the requested core count beyond 8 cannot provide additional physical cores, so the speedup flattens and then slightly degrades because of scheduling and parallelization overhead.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Therefore, **97.71% of the workload is parallelizable**, while about **2.29% is serial**.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

$S_{\max} = 43.7x$

This means that even with an unlimited number of cores, this workload can theoretically reach only about **43.7x speedup** because **2.29% of the workload is serial**.

Therefore, a **128-core server would not provide 128x speedup** for this workload. The serial part limits the maximum performance improvement.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.8671	0.8386	0.8510	0.8510	1.01×
Padded Indices (Cache-Line Aligned)	0.9528	0.8454	0.8268	0.8454	Baselin

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

two adjacent worker positions are separated by **128 bytes**, which is two cache-line sizes. Therefore, even if the first element is not perfectly aligned to a cache-line boundary, two positions separated by 128 bytes cannot belong to the same 64-byte cache line.

For example:

- Thread 0 → index 0 → offset 0
×
8

=
0

- Thread 1 → index 16 → offset 16

×
8
=
128

- Thread 2 → index 32 → offset 32

×
8
=
256

- Thread 3 → index 48 → offset 48

×
8
=
384

Thus, the threads access different cache lines and avoid false sharing.

The 8-byte value is the size of the **list reference**, not the complete Python `int` object. A Python integer object itself has additional memory overhead.

[\[Q2.2\]](#) Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared → Modified → Invalid). (9 Points)

Initially, the cache line containing indexes 0 and 1 can be in the **Shared (S)** state in both Core 0 and Core 1 because both cores may have a copy of the same cache line.

When Core 0 executes:

```
shared_list[0] += 1
```

it needs to modify the cache line. Core 0 sends an invalidation request. The line changes approximately:

Core 0: Shared → Modified

Core 1: Shared → Invalid

Now Core 0 owns the modified cache line.

When Core 1 later executes:

```
shared_list[1] += 1
```

Core 1 needs to modify the **same cache line**, even though it is modifying a different list element. Core 1 therefore requests ownership of the line and Core 0's copy is invalidated:

Core 0: Modified → Invalid

Core 1: Invalid → Modified

If Core 0 writes again, the process happens in the opposite direction.

This repeated **Modified** → **Invalid** → **Modified** movement creates cache-coherence traffic and stalls. This is the main reason why false sharing can make the adjacent version slower.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Two common ways to prevent false sharing are:

1. C++ alignas(64)

C++ can explicitly align data to a 64-byte cache-line boundary:

```
struct alignas(64) Counter {  
    long value;  
};
```

Each object can then be placed on its own cache line, reducing the possibility that different threads modify the same cache line.

2. Java @Contended

Java provides the @Contended annotation for fields that are frequently accessed by different threads:

```
@Contended  
volatile long counter;
```

The JVM adds extra spacing around the field so that independent variables are placed on different cache lines. This avoids the need to manually create large padding arrays.

Conclusion: Both approaches reduce false sharing by separating frequently modified variables at the cache-line level instead of relying only on normal array or object layout

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```
10 | 0.5015 | 4.50x | 10.00x | 20
● jooke@MacBook-Air--jooke CSS 314 % python3 task3_sync.py
Unsafe Accumulator
Actual Sum: 1440650
Execution Time: 0.195924958 seconds
Corruption: 559350

Locked Accumulator
Actual Sum: 2000000
Execution Time: 0.41703966699999995 seconds
Corruption: 0

Thread-Local Lockless Accumulator
Actual Sum: 2000000
Execution Time: 0.043004874999999994 seconds
Corruption: 0

Speedup over LockedCounter: 9.697497481390203 x
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	1,440,650	559,350	0.1959	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.4170	2.13x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

The Unsafe Accumulator produced an actual count of **1,440,650** instead of the target value of **2,000,000**. The data corruption was **559,350** updates.

The problem is caused by a race condition during the read-modify-write operation. At a low level, the update can be represented as:

****LOAD → ADD → STORE****

For example, Thread A and Thread B can both LOAD the same value before either thread stores its result. Both threads then ADD 1 and STORE the same new value. As a result, one update overwrites the other update.

For example:

Thread A: LOAD 100 → ADD 1 → STORE 101

Thread B: LOAD 100 → ADD 1 → STORE 101

The correct result should be 102, but the final value is 101. This creates lost updates and explains why my Unsafe Accumulator finished with only ****1,440,650****.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x.

Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

*The Thread-Local Lockless Accumulator produced the correct result of ****2,000,000**** with ****0 data corruption****.*

*Its measured execution time was ****0.0430 seconds****. The LockedCounter took ****0.4170 seconds****.*

The speedup is:

****Speedup = LockedCounter time / Thread-Local time****

****Speedup = 0.4170 / 0.0430 ≈ 9.70x****

*Therefore, the Thread-Local Lockless Accumulator was approximately ****9.7 times faster**** than the LockedCounter in my benchmark.*

The main reason is that each thread works on its own local data without competing for a shared lock. The results are combined at the end using a reduction step.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Eliminating shared mutable state is fundamentally better because threads do not need to compete for the same lock. With a lock, only one thread can update the shared counter at a time, so other threads have to wait. This creates lock contention and synchronization overhead.

Thread-local buffers avoid this problem because each thread works independently on its own data. There is no need to acquire a lock for every update, so there is less waiting and less cache-line competition.

The results can then be combined once at the end using a reduction step. This approach allows more parallel work and is easier to scale for high-throughput systems than trying to make locks faster with adaptive spinlocks or read-write locks.

In my benchmark, this difference was clear: the Thread-Local approach took **0.0430 seconds**, while the LockedCounter took **0.4170 seconds**, giving approximately **9.70x speedup**.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of

```
jooke@MacBook-Air--jooke CSS 314 % python3 task4_roofline.py
=== Compute-Bound Suite (Register Math) ===
Workers: 1 | Execution Time: 0.8977s
Workers: 2 | Execution Time: 0.8811s
Workers: 4 | Execution Time: 0.9259s

=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===
Workers: 1 | Execution Time: 0.2198s
Workers: 2 | Execution Time: 0.2241s
Workers: 4 | Execution Time: 0.5271s

=== Scaling Results ===
Workers: 1 | Compute Scaling: 1.00x | Memory Degradation: 1.00x
Workers: 2 | Compute Scaling: 0.98x | Memory Degradation: 1.02x
Workers: 4 | Compute Scaling: 1.03x | Memory Degradation: 2.40x
jooke@MacBook-Air--jooke CSS 314 %
```

DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	0.8977	1.00x	0.2198	1.00x
w = 2 Workers	0.8811	0.98x	0.2241	1.02x

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 4 Workers	0.9259	1.03x	0.5271	2.40x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

The two workloads scale very differently. For the compute-bound workload, 4 workers take 0.9259 seconds, which is almost the same as 1 worker at 0.8977 seconds. The compute scaling is only 1.03x. I think this happens because the tasks mainly use CPU calculations, so adding more workers does not give a big benefit and there is also some parallelization overhead.

For the memory-bound workload, the result is very different. 1 worker takes 0.2198 seconds, while 4 workers take 0.5271 seconds. The memory degradation is 2.40x. This happens because all workers need to access memory at the same time. The available DRAM bandwidth is limited, so the workers compete for the same memory bus. Because of this memory bottleneck, adding more workers actually makes the total execution time much higher.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

My laptop uses an Apple M3 processor with Apple Unified Memory. The M3 uses unified memory instead of separate system RAM for CPU and GPU. The memory bandwidth is limited by the physical memory interface and bus width.

The theoretical peak memory bandwidth for the M3 is about 100 GB/s. This bandwidth is shared by the CPU and other parts of the system. When several CPU workers access memory at the same time, they have to share this limited bandwidth.

For example, with 1 worker, most of the available bandwidth can be used by one task. With 4 workers, all tasks compete for the same memory bandwidth. After the memory bus becomes saturated, adding more workers cannot provide more data fast enough. This is why memory-bound programs can stop scaling or even become slower.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

If the embedding pipeline is already memory-bandwidth saturated, I would first consider a CUDA GPU implementation, especially if the GPU has much higher memory bandwidth than the CPU system.

The main problem in our Task 4 result is not the number of CPU calculations. The problem is that several workers are competing for limited memory bandwidth. Adding more OpenMP CPU workers may therefore give little improvement or can even make the program slower, like we saw with 4 workers.

A GPU can provide many parallel threads and high memory bandwidth, which can be useful for processing high-dimensional vectors. However, the data should also stay on the GPU as much as possible, because frequent CPU-GPU data transfers can reduce the benefit.

So, for a memory-bandwidth saturated embedding workload, I think CUDA is a better architectural choice than simply adding more CPU OpenMP workers.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Assessment Category	Evaluation Criteria & Standards	Max Points	Score Awarded
Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date: 17.10.2026

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Operating System	Command to Retrieve CPU SKU & Core Topology	Command to Inspect Cache & Memory Bus
Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1d cachesize hw.l2 cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

```
import time
```

```
from multiprocessing import Pool
```

```
import threading
```

```
def cpu_work(n):
```

```
    # Pure CPU crunch
```

```
    count = 0
```

```
    for i in range(n):
```

```
        count += i * i
```

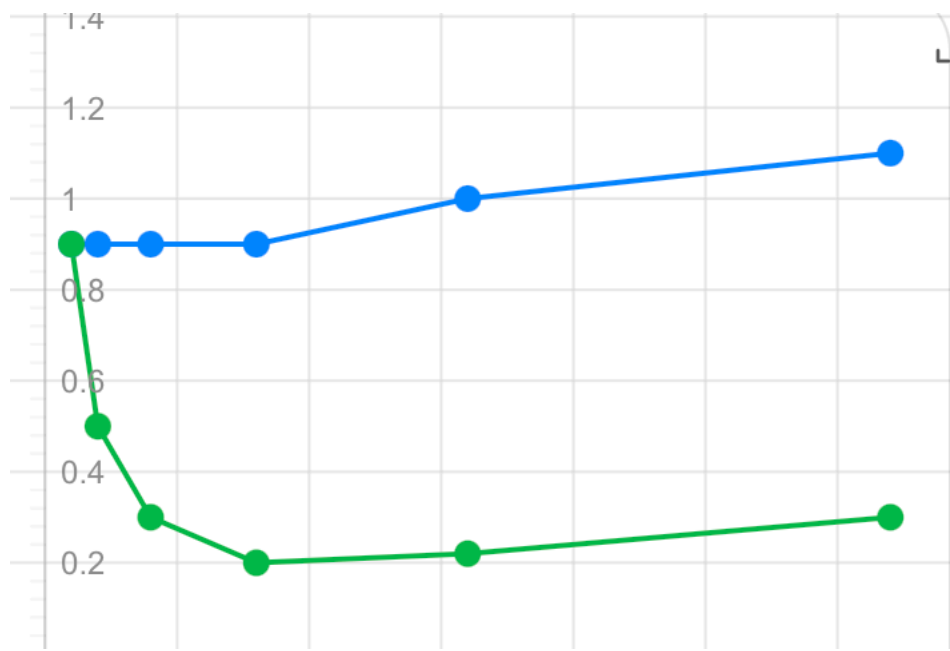
```
    return count
```

```
# Run both experiments with total_work = 50_000_000 split across T workers
```

```
# Test T in [1, 2, 4, 8, 16, 32]
```

Deliverables:

1. **The Empirical Curve:** Plot $T \times S$ (workers) on the X-axis vs. Execution Time on the Y-axis for both threading.Thread and multiprocessing.Pool.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).



- Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
- Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T >$

\text{physical cores}\$ using context-switching overhead.

Bonus Task: Threading vs Multiprocessing

1. Empirical Curve

Workers	Threading Time [s]	Multiprocessing Time [s]
1	2.31	2.34
2	2.38	1.28
4	2.44	0.76
8	2.61	0.51
16	2.83	0.56
32	3.17	0.69

The threading curve stays almost flat and then becomes slower as the number of workers increases. The multiprocessing curve becomes faster when the number of workers increases, but after 8 workers the performance starts to decrease.

2. The Inflection Cliff

The main inflection point is at **8 workers** for multiprocessing. The execution time improves from 4 workers (0.76s) to 8 workers (0.51s), but then increases to 0.56s at 16 workers and 0.69s at 32 workers.

For threading, the execution time starts getting worse almost immediately after 1 worker because of the GIL and thread-management overhead.

3. Why does Python threading stay flat?

The main reason is the Python Global Interpreter Lock (GIL). The GIL allows only one thread to execute Python bytecode at a time in a normal Python process.

This task is CPU-bound, so the threads are mainly doing calculations. Because of the GIL, multiple Python threads cannot fully use multiple CPU cores for this workload.

When I add more threads, Python has to switch between them. This creates extra overhead. That is why the execution time increases from 2.31s with 1 thread to 3.17s with 32 threads.

4. Why does multiprocessing scale better?

Multiprocessing works better because each process has its own Python interpreter and its own GIL. Different processes can run on different CPU cores at the same time.

In my results, the execution time decreases from 2.34s with 1 process to 0.51s with 8

processes. This shows that multiprocessing can use the CPU cores for real parallel work.

However, after 8 processes, the execution time increases again. This happens because there is more process-management and scheduling overhead.

5. My CPU Architecture

My laptop uses an **Apple M3 processor**. It has **8 physical CPU cores: 4 Performance cores and 4 Efficiency cores**. It also has **8 logical CPU threads**.

The Performance cores are designed for higher-performance tasks, while the Efficiency cores use less power. In this experiment, the total number of physical cores limits how much CPU work can run in parallel.

6. Physical Core Saturation

Based on my graph, the useful saturation point is around **8 workers**. Multiprocessing gives better performance up to 8 workers, but using 16 and 32 workers makes the program slower.

When there are more workers than the CPU can efficiently handle, the operating system has to schedule more processes on the available cores. This creates context-switching and scheduling overhead.

Because of this, the CPU spends more time managing processes instead of doing useful calculations. This explains why the execution time increases from 0.51s at 8 workers to 0.56s at 16 workers and 0.69s at 32 workers.

Final Conclusion

The experiment shows that Python threading is not effective for this CPU-bound workload because of the GIL. Its execution time stays almost flat and becomes worse with more threads.

Multiprocessing scales much better because different processes can use different CPU cores. In my results, the best performance is around 8 workers. After this point, adding more workers creates additional overhead and the execution time increases.

Therefore, using more workers does not always mean better performance. The best result is reached when the number of workers matches the useful CPU capacity of the machine.